

Data_Cleaning

June 23, 2026

1 Capitolo 2: Data Cleaning e Standardizzazione

I JSON scaricati sono pieni di HTML sporco e i nomi delle fonti energetiche sono un casino (es. a volte il carbone è “coal”, altre “lignite”). Qui facciamo due cose: 1. Usiamo BeautifulSoup per estrarre solo i valori numerici. 2. Mappiamo le decine di varianti testuali in 11 macro-categorie fisse. In questo modo avremo un dataset pulito e confrontabile tra le varie nazioni.

```
[ ]: import os
import json
import re
import pandas as pd
from bs4 import BeautifulSoup

[ ]: def categorize_source(raw_name):
    """ Mappa le fonti grezze nelle 11 categorie standard """
    name = raw_name.lower()

    if 'wind' in name: return 'source_wind_total_perc'
    if 'coal' in name or 'lignite' in name: return 'source_coal_total_perc'
    if 'solar' in name: return 'source_solar_total_perc'
    if 'hydro' in name: return 'source_hydro_total_perc'
    if 'gas' in name: return 'source_gas_total_perc'
    if 'biomass' in name: return 'source_biomass_total_perc'
    if 'oil' in name: return 'source_oil_total_perc'
    if 'geothermal' in name: return 'source_geothermal_total_perc'
    if 'nuclear' in name: return 'source_nuclear_total_perc'
    if 'waste' in name: return 'source_waste_total_perc'

    return 'source_other_total_perc'
```

1.0.1 1.0.1 Parsing e Creazione Master Dataset

Questo script itera sui JSON grezzi. Estrae CO2, % rinnovabili e TWh totali. Per il mix energetico, usa una regex per separare dinamicamente il nome della fonte dal valore percentuale. Riempie i vuoti con 0.0, ordina tutto per nazione/mese e butta fuori il `master_dataset_clean.csv`.

```
[ ]: input_dir = '../data/raw/'
output_file = '../data/processed/master_dataset_clean.csv'
```

```

nations_dir = '../data/processed/nations'
all_records = []

os.makedirs(nations_dir, exist_ok=True)

for filename in os.listdir(input_dir):
    if not filename.endswith('.json'):
        continue

    country_name = filename.replace('nowtricity_raw_', '').replace('.json', '').
    title()
    filepath = os.path.join(input_dir, filename)

    with open(filepath, 'r', encoding='utf-8') as file:
        dati_lista = json.load(file)

    for item in dati_lista:
        month_id = item.get('mese', 0)
        if month_id == 0:
            continue

        payload = item.get('json_payload', {})
        record = {
            'country_name': country_name,
            'month_id': month_id
        }

        html_content = payload.get('html', '')
        if not html_content:
            continue

        soup = BeautifulSoup(html_content, 'html.parser')

        # Estrazione delle metriche principali: CO2, quota rinnovabili e
        produzione totale (TWh)
        co2_raw = soup.find("text", class_="emissions")
        if co2_raw:
            co2_str = co2_raw.text.strip().replace("g", "").strip()
            record["co2_g_kwh"] = float(co2_str) if co2_str.replace('.', '', 1).
            isdigit() else None

        renewable_raw = soup.find("div", class_="emissions-graph-renewable")
        if renewable_raw:
            renewable_str = renewable_raw.text.strip().replace("% renewable",
            "").strip()
            record["renewable_perc"] = float(renewable_str) if renewable_str.
            replace('.', '', 1).isdigit() else None

```

```

total_raw = soup.find("div", class_="emissions-graph-total")
if total_raw:
    total_str = total_raw.text.strip().replace("TWh total", "").strip()
    record["total_twh"] = float(total_str) if total_str.replace('.', '')
    ↳'', 1).isdigit() else None

energy_entries = soup.find_all("div", class_="entry")

for entry in energy_entries:
    text = entry.text.strip()
    match = re.match(r"(.*)\s*\(((\d\.)+)%\)", text)

    if match:
        raw_name = match.group(1).strip()
        perc_val = float(match.group(2))

        clean_col_name = raw_name.replace(' ', '_').replace('(', '').
    ↳replace(')', '')
        detail_col_name = f"source_{clean_col_name}_detail_perc"
        record[detail_col_name] = perc_val

        macro_col_name = categorize_source(raw_name)
        record[macro_col_name] = record.get(macro_col_name, 0.0) +
    ↳perc_val

    all_records.append(record)

# Creazione del DataFrame e normalizzazione dei valori nulli
df_clean = pd.DataFrame(all_records)
df_clean.fillna(0.0, inplace=True)

# Ordino e resetto l'indice
df_clean = df_clean.sort_values(['country_name', 'month_id']).
    ↳reset_index(drop=True)
df_clean['source_hydro_total_perc'] = df_clean['source_hydro_total_perc'].
    ↳round(1)

df_clean.to_csv(output_file, index=False)

for nation, df_nation in df_clean.groupby('country_name'):
    clean_filename = nation.replace(" ", "_").lower()
    nation_path = os.path.join(nations_dir, f"{clean_filename}.csv")
    df_nation.to_csv(nation_path, index=False)

print(f"Estrazione completata. Processati {len(df_clean)} record.")

```

1.0.2 1.0.2 Sanity Check

Se il mix delle fonti non somma a ~100%, i dati sono da buttare. Questo blocco verifica la somma orizzontale delle 11 macro-categorie per ogni riga, applicando una tolleranza tra 99.0% e 101.0% per gestire i normali arrotondamenti fatti dal sito sorgente. Se una riga sfiora, viene segnalata.

```
[ ]: # Lista definitiva delle macro-colonne che devono sommare a ~100%
macro_cols = [
    'source_biomass_total_perc', 'source_gas_total_perc',
    ↪ 'source_hydro_total_perc',
    'source_solar_total_perc', 'source_waste_total_perc',
    ↪ 'source_nuclear_total_perc',
    'source_oil_total_perc', 'source_geothermal_total_perc',
    ↪ 'source_coal_total_perc',
    'source_wind_total_perc', 'source_other_total_perc'
]

# Caricamento del master dataset appena generato
df_test = pd.read_csv('../data/processed/master_dataset_clean.csv')

# Controllo di sicurezza: garantisce che tutte le macro-colonne esistano nel
↪ DataFrame (riempie i mancanti con 0.0)
for col in macro_cols:
    if col not in df_test.columns:
        df_test[col] = 0.0

# Assicuro che tutte le colonne esistano prima di sommare
df_test['verification_sum'] = df_test[macro_cols].sum(axis=1)

# Filtro le anomalie fuori dal range 99-101%
anomalies = df_test[(df_test['verification_sum'] < 99.0) |
↪ (df_test['verification_sum'] > 101.0)]

print(f"Totale righe analizzate: {len(df_test)}")
print(f"Anomalie rilevate: {len(anomalies)}")

if len(anomalies) > 0:
    print("\n[!] ATTENZIONE: Le seguenti righe non hanno superato il controllo
↪ di integrità del 100%:")
    print(anomalies[['country_name', 'month_id', 'verification_sum']].head(15))
else:
    print("\n[V] SUCCESSO: Tutte le righe sommano al 100%. Il dataset è
↪ matematicamente coerente.")
```